

مستند معماری پلتفرم گلدکسا

نسخه: 1.0 تاریخ: 1405/02/15

فهرست مطالب

- 1.1. مقدمه
- 1.2. هدف مستند
- 1.3. مخاطبین
- 2.1. نیازمندی‌های عملکردی
- 2.2. نیازمندی‌های غیرکارکردی
- 3.1. دید کلی معماری
- 3.2. خلاصه رویکرد
- 3.3. چرایی انتخاب معماری
- 4.1. لایه‌ها و مؤلفه‌ها
- 4.2. Presentation
- 4.3. Application
- 4.4. Domain
- 4.5. Infrastructure
- 5.1. نمودارهای معماری
- 5.2. Component Diagram
- 5.3. Data Flow Diagram
- 5.4. Deployment Diagram
- 6.1. فناوری‌ها و ابزارها
- 6.2. زبان‌ها و فریم‌ورک‌ها
- 6.3. پایگاه داده و کش
- 6.4. Messaging و Event Bus
- 6.5. ابزارهای CI/CD و مانیتورینگ
- 7.1. الزامات غیرکارکردی
- 7.2. مقیاس پذیری
- 7.3. در دسترس پذیری
- 7.4. امنیت
- 7.5. پایش و لاگینگ
- 8.1. مسیر مهاجرت و نقشه راه
- 8.2. فازهای توسعه
- 8.3. معیارهای جداسازی سرویس‌ها
- 9.1. پیوست‌ها
- 9.2. واژه‌نامه
- 9.3. منابع

1. مقدمه

1.1. هدف مستند

هدف این سند ارائه معماری فنی پلتفرم گلدکسا و مستندسازی تصمیمات معماری سیستم است تا تیم توسعه، DevOps و ذی‌نفعان فنی درک مشترکی از ساختار سیستم داشته باشند.

این مستند چارچوب معماری، فناوری‌های انتخاب شده، لایه‌های سیستم و مسیر توسعه آینده سیستم را توضیح می‌دهد.

1.2. دامنه

این مستند معماری سطح بالا (High Level Architecture) سیستم گلدکسا را پوشش می‌دهد. جزئیات پیاده‌سازی در مستندات فنی جداگانه ارائه خواهند شد.

1.3. مخاطبین

مخاطبین این مستند شامل موارد زیر هستند:

- تیم توسعه Backend
- تیم توسعه Frontend
- تیم DevOps
- معماران نرم‌افزار
- مدیران فنی پروژه

2. نیازمندی‌ها

2.1. نیازمندی‌های عملکردی

پلتفرم گلدکسا باید قابلیت‌های زیر را ارائه دهد:

- مدیریت کاتالوگ محصولات
- محاسبه قیمت محصولات
- مدیریت سفارش‌ها
- احراز هویت کاربران
- مدیریت کاربران و دسترسی‌ها
- پنل مدیریت برای مدیریت سیستم
- اتصال به درگاه‌های پرداخت
- ارسال اعلان‌ها و پیام‌ها

2.2. نیازمندی‌های غیرکارکردی

سیستم باید دارای ویژگی‌های زیر باشد:

- پشتیبانی از حدود 1000 کاربر همزمان
- توان پردازش حدود 50 درخواست در ثانیه

- در دسترس پذیری 99.9 درصد
 - قابلیت مقیاس پذیری افقی
 - امنیت بالا در انتقال و ذخیره داده
 - قابلیت مانیتورینگ و لاگ گیری
-

3. دید کلی معماری

3.1. خلاصه رویکرد

در فاز اول پروژه از معماری Modular Monolith مبتنی بر اصول Clean Architecture و Hexagonal Architecture استفاده می شود.

در این مدل:

- ماژول های مختلف سیستم به صورت منطقی از یکدیگر جدا هستند اما در قالب یک برنامه واحد اجرا می شوند.
- هسته دامنه سیستم از وابستگی به دیتابیس، فریم ورک ها و زیرساخت جدا نگه داشته می شود.
- ارتباط با زیرساخت از طریق Port و Adapter انجام می شود.

3.2. چرایی انتخاب این معماری

دلایل انتخاب این معماری برای پروژه گلدکسا:

- پیاده سازی سریع برای MVP
 - کاهش پیچیدگی زیرساخت نسبت به Microservices
 - تست پذیری بالا
 - نگهداری آسان تر کد
 - امکان مهاجرت تدریجی به معماری Microservices
-

4. لایه‌ها و مؤلفه‌ها

4.1. لایه Presentation

این لایه شامل رابط‌های کاربری سیستم است:

- وب اپلیکیشن با Next.js
- پنل مدیریت
- اپلیکیشن موبایل در فازهای بعدی

4.2. لایه Application

این لایه شامل Use Case های سیستم است.

نمونه Use Case ها:

- ثبت سفارش
- محاسبه قیمت
- مدیریت کاربران
- مدیریت پرداخت

4.3. لایه Domain

هسته اصلی سیستم در این لایه قرار دارد.

شامل:

- Entity ها
- Value Object ها
- Domain Services
- Repository Interface ها

4.4. لایه Infrastructure

این لایه پیاده‌سازی وابستگی‌های خارجی سیستم را فراهم می‌کند.

نمونه‌ها:

- اتصال به PostgreSQL
 - اتصال به Redis
 - اتصال به سرویس‌های پرداخت
 - اتصال به سرویس‌های پیام
-

5. نمودارهای معماری

5.1 Component Diagram

سیستم شامل ماژول‌های زیر است:

- Auth Module
- Catalog Module
- Pricing Module
- Order Module
- Notification Module
- Admin Module

5.2 Data Flow Diagram

جریان داده در سیستم به شکل زیر است:

User -> Frontend -> API -> Application Layer -> Domain -> Database

5.3 Deployment Diagram

ساختار استقرار سیستم:

- Backend برای Docker Container
 - Frontend برای Docker Container
 - PostgreSQL Database
 - Redis Cache
-

6. فناوری‌ها و ابزارها

6.1. زبان‌ها و فریم‌ورک‌ها

:Backend

- Node.js
- TypeScript
- Express یا NestJS

:Frontend

- Next.js
- React

6.2. پایگاه داده و کش

- PostgreSQL به عنوان پایگاه داده اصلی
- Redis برای کش

6.3. Messaging

در فازهای بعدی استفاده از Kafka یا AWS SQS

6.4. ابزارهای CI/CD و مانیتورینگ

- Docker
 - GitHub Actions
 - Prometheus
 - Grafana
 - ELK Stack
-

7. الزامات غیرکارکردی

7.1. مقیاس پذیری

استفاده از Docker و Kubernetes برای مقیاس پذیری افقی

7.2. در دسترس پذیری

هدف سیستم دستیابی به SLA برابر 99.9 درصد است.

7.3. امنیت

- TLS برای ارتباطات شبکه
- JWT برای احراز هویت
- رمزنگاری داده ها در پایگاه داده

7.4. پایش و لاگینگ

استفاده از سیستم مانیتورینگ و جمع آوری لاگ برای تشخیص سریع خطاها

8. مسیر مهاجرت و نقشه راه

8.1. فازهای توسعه

فاز اول:

- Modular Monolith

فاز دوم:

- تفکیک سرویس Pricing
- تفکیک سرویس Catalog

فاز سوم:

- تبدیل Order و Auth به Microservices

- افزودن API Gateway

8.2. معیارهای جداسازی سرویس‌ها

- افزایش ترافیک مازول
- نیاز به مقیاس مستقل
- تیم توسعه مستقل

9. پیوست‌ها

9.1. واژه‌نامه

- Monolith: معماری نرم‌افزاری که در آن کل برنامه به عنوان یک واحد واحد توسعه یافته، مستقر و مقیاس‌بندی می‌شود.
- Microservice: سبکی معماری نرم‌افزاری که در آن یک برنامه به عنوان مجموعه‌ای از خدمات کوچک و مستقل ساخته می‌شود.
- Domain: بخش خاصی از کسب و کار که سیستم نرم‌افزاری برای آن طراحی شده است.
- Use Case: توصیفی از تعامل کاربر با سیستم برای دستیابی به یک هدف خاص.

9.2. منابع

- AWS Architecture Documentation
- Clean Architecture by Robert C. Martin
- Microservices Patterns by Chris Richardson